

Университет ИТМО
Факультет программной инженерии и компьютерной техники

Системное и прикладное программное обеспечение
Программно-информационные системы

Отчет
о производственной, технологической практике
по теме «Разработка модуля
контекстно-зависимого автодополнения
запросов на YQL»

Автор: Смирнов Виктор Игоревич, Р34131

Руководитель от университета ИТМО:

Маркина Татьяна Анатольевна, старший преподаватель

Руководитель от профильной организации:

Логинов Иван Павлович, генеральный директор
ООО «Специальные Средства Программирования»

Санкт-Петербург, 2025

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	4
ХАРАКТЕРИСТИКА ОРГАНИЗАЦИИ	5
ВВЕДЕНИЕ	6
1 Сбор и анализ требований	7
1.1 Требования к пользовательскому интерфейсу	7
1.2 Функциональные требования	7
1.3 Технические требования	7
2 Реализация автодополнения ключевых слов	8
3 Проектирование интерфейса источника имен объектов БД	11
4 Реализация автодополнения имен объектов БД	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

СУБД — система управления базами данных

YDB — распределённая отказоустойчивая Distributed SQL СУБД

SQL — декларативный язык программирования для получения, создания, модификации и управления данными в реляционной базе данных

YQL — универсальный декларативный язык запросов к YDB, диалект SQL

CLI — command-line interface

ANTLR — генератор парсеров по грамматике

GitHub — веб-сервис для хостинга IT-проектов и их совместной разработки

ХАРАКТЕРИСТИКА ОРГАНИЗАЦИИ

Основным видом деятельности общества с ограниченной ответственностью «Специальные Средства Программирования» являются научные исследования и разработки в области естественных и технических наук.

ВВЕДЕНИЕ

Целью практики по теме «Разработка модуля контекстно-зависимого автодополнения запросов на YQL» является повышение качества автодополнения запросов на YQL в YDB CLI.

Изначально автодополнение в данном клиентском приложении практически отсутствовало. Необходимо было сперва подготовить индивидуальное задание на практику, собрать и проанализировать требования к будущему решению, поставить задачу, выбрать подходящие технологии для реализации, разобраться в процессе разработки (сборка проекта, интеграция с другими модулями, наладить взаимодействие со смежными командами), а далее инкрементально решать поставленную задачу.

1 Сбор и анализ требований

Первый этап разработки включал в себя сбор и анализ требований к модулю автодополнения YQL и их представление в текстовом виде. Критерием выполнения являлось создание задач в сервисе GitHub в проектах YDB и YQL и описание требований в комментариях.

На данном этапе были налажена связь с представителями команды разработки YDB SDK, которые отвечают за YDB CLI. У них не было конкретных сформулированных требований, так что их пришлось выяснять в процессе переговоров.

1.1 Требования к пользовательскому интерфейсу

Были выделены следующие требования к пользовательскому интерфейсу:

- (UIR1) в интерактивном режиме YDB CLI должно осуществляться автодополнение запросов на YQL;
- (UIR2) по нажатию на клавишу TAB на экране должен быть отображен список кандидатов на автодополнение.

1.2 Функциональные требования

Были выделены следующие функциональные требования:

- (FR1) автодополнение должно исключать кандидатов, не подходящих по синтаксическому контексту;
- (FR2) автодополнение должно предлагать именованные объекты базы данных.

1.3 Технические требования

Были выделены следующие технические требования:

- (TR1) модуль автодополнения должен быть реализован на языке программирования C++;
- (TR2) сборка модуля автодополнения должна осуществляться при помощи системы Yatool;
- (TR3) автодополнение должно завершать работу не более чем за 0.5 секунд.

Приведенные выше требования были описаны в комментариях к задачам. Как только требования были согласованы, можно было продолжить работу в заданном направлении.

2 Реализация автодополнения ключевых слов

Данный этап включает в себя реализацию автодополнения ключевых слов в модуле автодополнения YQL и интеграция модуля в приложение YDB CLI. Критерием выполнения данного этапа является создание модуля автодополнения YQL, реализованный алгоритм возвращающий ключевые слова в качестве кандидатов на автодополнение, автодополнение ключевых слов в YDB CLI в интерактивном режиме.

В качестве основы автодополнения была выбрана библиотека antlr4-c3. Она принимает на вход текст и грамматику языка в формате ANTLR4, а возвращает список лексем, которые ожидает парсер в заданной позиции.

Конечно, необходимо было скрыть библиотеку за интерфейсом движка автодополнения.

```
struct TCompletionInput {
    TStringBuf Text;
    size_t CursorPosition = Text.length();
};

enum class ECandidateKind {
    Keyword,
    ...
};

struct TCandidate {
    ECandidateKind Kind;
    TString Content;
    ...
};

struct TCompletion {
    ...
    TVector<TCandidate> Candidates;
};

class ISqlCompletionEngine {
public:
    ...
    virtual TCompletion Complete(TCompletionInput input) = 0;
    ...
};
```

Листинг 1 — Интерфейс движка автодополнения

Для выдачи ключевых слов достаточно было удалить лексемы, соответствующие, например, числам и именам. Кроме того, antlr4-c3 выдает сразу последовательности лексем, если последовательность однозначна.

```
TLocalSyntaxContext::TKeywords SiftedKeywords(  
    const TC3Candidates& candidates) {  
    const auto& vocabulary = Grammar->GetVocabulary();  
    const auto& keywordTokens = Grammar->GetKeywordTokens();  
  
    TLocalSyntaxContext::TKeywords keywords;  
    for (const auto& token : candidates.Tokens) {  
        if (keywordTokens.contains(token.Number)) {  
            auto display = Display(vocabulary, token.Number);  
            auto& following = keywords[std::move(display)];  
            for (auto next : token.Following) {  
                following.emplace_back(Display(vocabulary, next));  
            }  
        }  
    }  
    return keywords;  
}
```

Листинг 2 — Просеивание ключевых слов

Также необходимо было решить следующую проблему. Дело в том, что в SQL ключевые слова не являются полностью зарезервированными, из-за чего могут являться корректными именами, что отражено в грамматике и из-за чего antlr4-c3 возвращал почти все ключевые слова в каждой позиции, где ожидалось имя. Проблема была решена правильной настройкой antlr4-c3 – игнорированием некоторых правил вывода.

```
const TVector<TRuleId> KeywordRules = {  
    RULE(Keyword),  
    RULE(Keyword_expr_uncompat),  
    RULE(Keyword_table_uncompat),  
    RULE(Keyword_select_uncompat),  
    RULE(Keyword_alter_uncompat),  
    RULE(Keyword_in_uncompat),  
    RULE(Keyword_window_uncompat),  
    RULE(Keyword_hint_uncompat),  
    RULE(Keyword_as_compat),  
    RULE(Keyword_compat),  
};
```

Листинг 3 — Правила вывода, подлежащие игнорированию

Далее модуль автодополнения был внедрен в YDB CLI. В качестве цвета для ключевых слов был выбран синий (в соответствии с темой

Мопасо). Сделать это было не очень сложно, ведь для реализации интерактивного решения в клиентском приложении используется библиотека Replxx, предоставляющая интерфейс для интеграции автодополнения и подсветки.

Фрагменты кода приведены в приложениях к отчету.

3 Проектирование интерфейса источника имен объектов БД

Автодополнение ключевых слов уже заметно повысило пользовательский опыт разработки запросов на YQL в YDB CLI, но этого не достаточно для удобной работы. В любой удобной IDE реализованы подсказки также и имен различных сущностей, что очень ценят программисты, так как имена могут быть длинные, из-за чего их долго набирать, причем высок риск появления опечаток, а еще такие имена часто забываются.

Автодополнение имен является непростой задачей, так как требует также реализацию методов семантического анализа запроса с целью определения типа имени, требуемого в заданной позиции. Кроме того, необходим некоторый реестр имен.

Этот этап был посвящен проектированию интерфейса источника имен объектов БД для их получения в модуле автодополнения. Критерием выполнения была готовность спроектированного программного интерфейса источника имен объектов БД для модуля автодополнения YQL, разработана in-memory реализация, служащая заглушкой.

К интерфейсу источника имен были выдвинуты следующие требования:

- Источник имен должен допускать реализацию через вызов удаленной процедуры;
- Источник имен должен поддерживать фильтрацию имен по заданным ограничениям;
- Источник имен должен поддерживать фильтрацию имен по содержимому;
- Источник имен должен ограничивать размер списка имен;
- Источник имен должен выдавать отсортированные по релевантности имена.

Для модульного тестирования данного интерфейса также была разработана заглушка со списком имен, представленном прямо в коде.

Фрагменты кода приведены в приложениях к отчету.

4 Реализация автодополнения имен объектов БД

TODO: ...

ЗАКЛЮЧЕНИЕ

TODO: ...

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] A. V. Aho, M. S. Lam, R. Sethi, и J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2008.
- [2] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.